Computer Organization & Architecture

Unit- 11
Multiprocessors

Prepared By: Sweetu D. Sureja
Information Technology Department
V.V.P. Engineering College, Rajkot.

Characteristics of Multiprocessors

- The **term "processor"** In multiprocessor can mean either a central processing unit (CPU) or an input-output processor (lOP).
- A multiprocessor system is an **inter connection of two or more CPUs** with memory and input-output equipment.
- Multiprocessors are classified as **multiple instruction stream, multiple data stream (MIMD)** systems

- **Multiprocessing improves the reliability of the system** so that a **failure or error** in one part has a limited effect on the rest of the system.
- If a fault causes one processor to fail, a second processor can be assigned to perform the functions.

The benefit derived from a multiprocessor organization is an improved system performance by making

1. Multiple independent jobs can be made to operate in parallel.
2. A single job can be partitioned into multiple parallel tasks

Characteristics of Multiprocessors

- Multiprocessors are classified by the way their memory is organized.
- A multiprocessor system **with common shared memory is classified as a tightly coupled multiprocessor.**
- A multiprocessor system **with its own private local memory is classified as loosely coupled system**

Tightly Coupled System	Loosely Coupled System
Tasks and/or processors communicate in a highly synchronized fashion.	Tasks or processors do not communicate in a synchronized fashion.
Communicates through a common shared memory.	Communicates by message passing packets.
Shared memory system.	Distributed memory system.
Overhead for data exchange is lower comparatively.	Overhead for data exchange is higher comparatively.

Interconnection Structures

- When a system has one or more CPUs and one or more memory units, we need a proper way to connect them so they can communicate and share data.
  This connection system is called an **interconnection structure**.
- This connection method is divided into **5 types**.

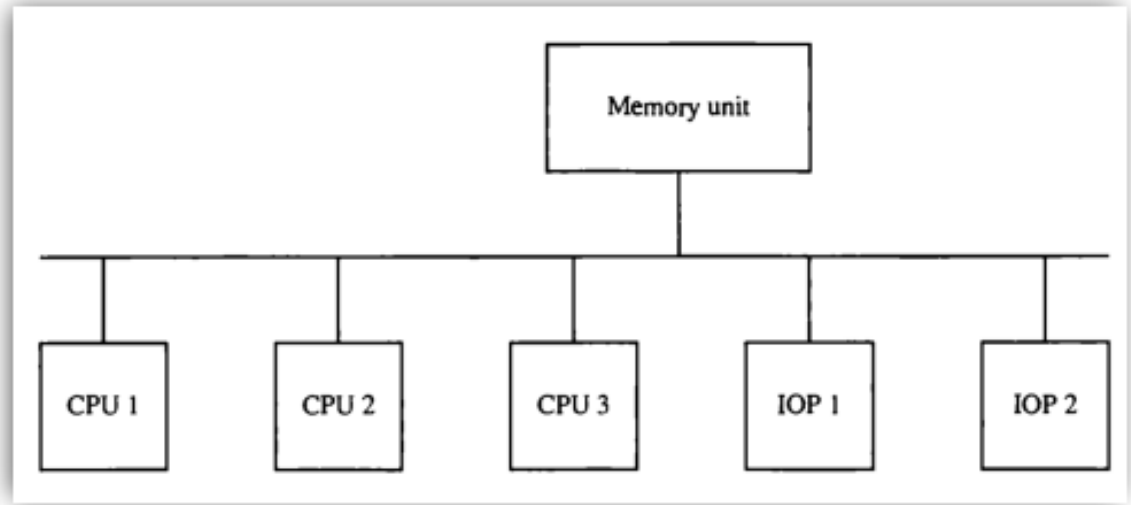
There are several physical forms available for establishing an interconnection network:

1. Time-shared common bus.
2. Multiport memory
3. Crossbar switch
4. Multistage switching network.
5. Hypercube system

Interconnection Structures

1) Time-shared common bus Interconnection Structures

- A common bus multiprocessor system consists of a number of processors connected through a common path to a memory unit.
- Only one processor can communicate with the memory or another processor at any given time.
- Transfer operations are conducted by the processor that is in control of the bus at the time.



- Any other processor wishing to initiate a transfer must first determine the availability status of the bus and only after the bus becomes available, the processor address the destination unit to initiate the transfer.
- A single common-bus system is restricted to one transfer at a time.
- This means that when one processor is communicating with the memory, all other processors are either busy with internal operations or must be idle waiting for the bus.

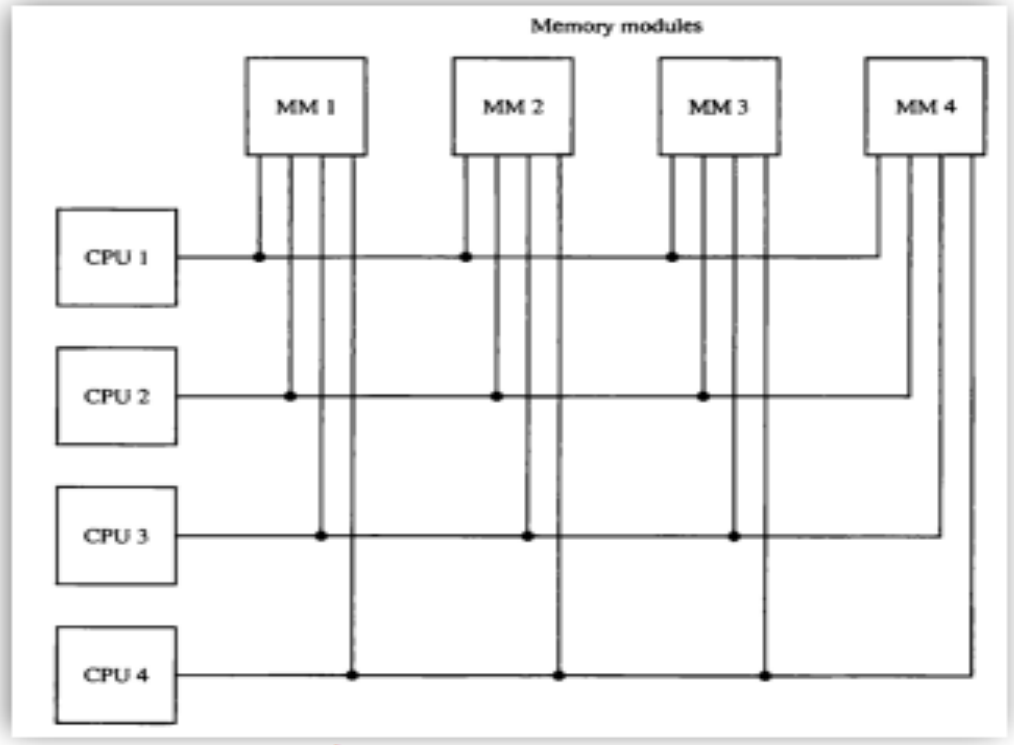
❖ Suppose at this time CPU 1 is connected with memory unit , at the same time CPU 3 have needed memory unit then CPU 3 is in waiting mode whenever CPU 3 time is started then after it will access memory unit.

❖ All this doing smoothly by using the Bus controller.

Interconnection Structures

2) Multiport Memory Interconnection Structures

- A multiport memory system employs separate buses between each memory module and each CPU.
- Each processor bus is connected to each memory module.
- A processor bus consists of the address, data, and control lines required to communicate with memory.



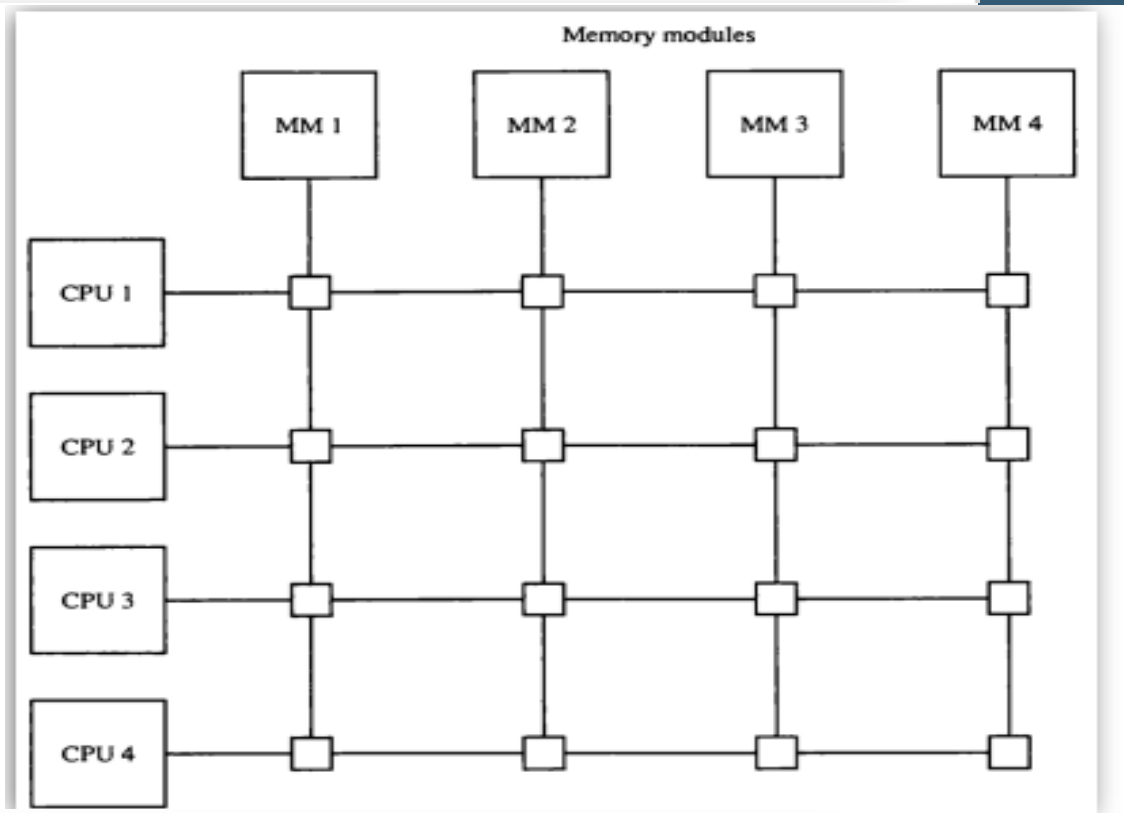
Suppose CPU 1 is communicating with the memory 1 and at same time Also CPU 3 also wants to communicating memory 1 then which Connection is communicating first ? It will decides depends on priority .

- Memory access conflicts are resolved by assigning fixed priorities to each memory port.
- Thus CPU 1 will have priority over CPU 2, CPU 2 will have priority over CPU 3, and CPU 4 will have the lowest priority.
- The advantage of the multiport memory organization is the high transfer rate that can be achieved because of the multiple paths between processors and memory.
- The disadvantage is that it requires expensive memory control logic and a large number of cables and connectors.

Interconnection Structures

3) Crossbar Switch Interconnection Structures

- The small square in each cross point is a switch that determines the path from a processor to a memory module.
- Each switch point has control logic to set up the transfer path between a processor and memory.



- The circuit consists of multiplexers that select the data, address, and control from one CPU for communication with the memory module.

If CPU 1 is connected with memory 1 then switch done the cross sign it means another cpu can't be communicate With memory 1

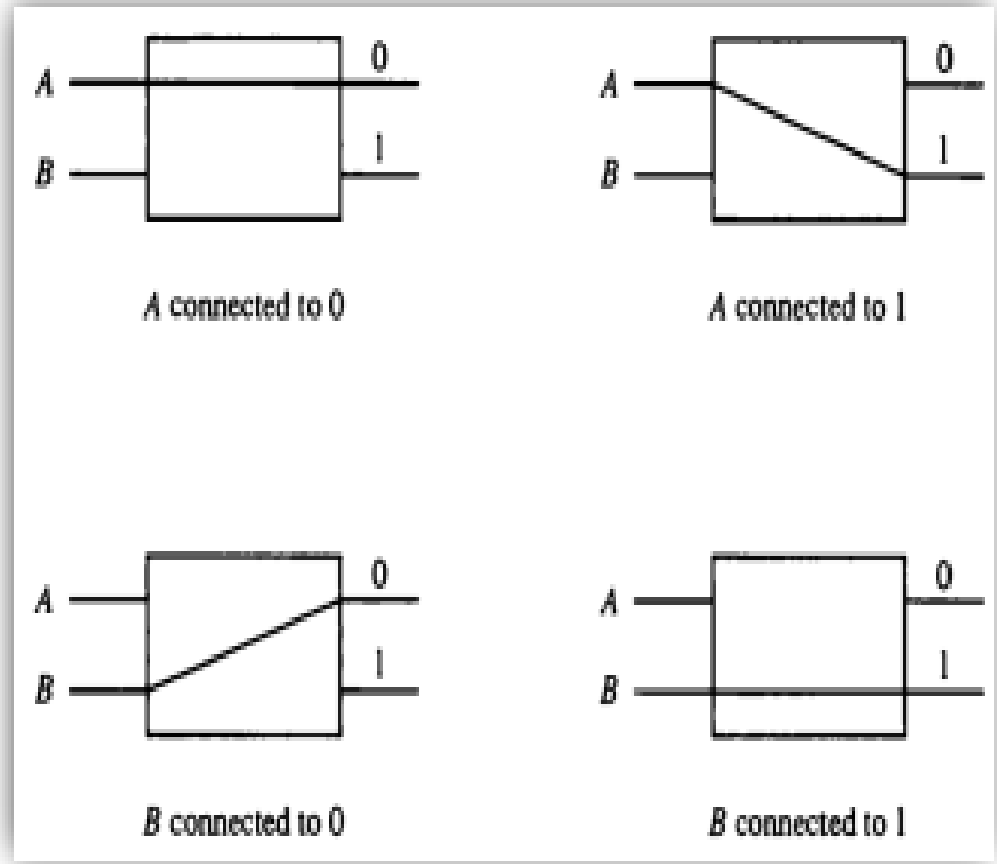
- A crossbar switch organization supports simultaneous transfers from memory modules because there is a separate path associated with each module.
- However, the hardware required to implement the switch can becomes quite large and complex.

Cont...

Interconnection Structures

4) Multistage Switching Network Interconnection Structures

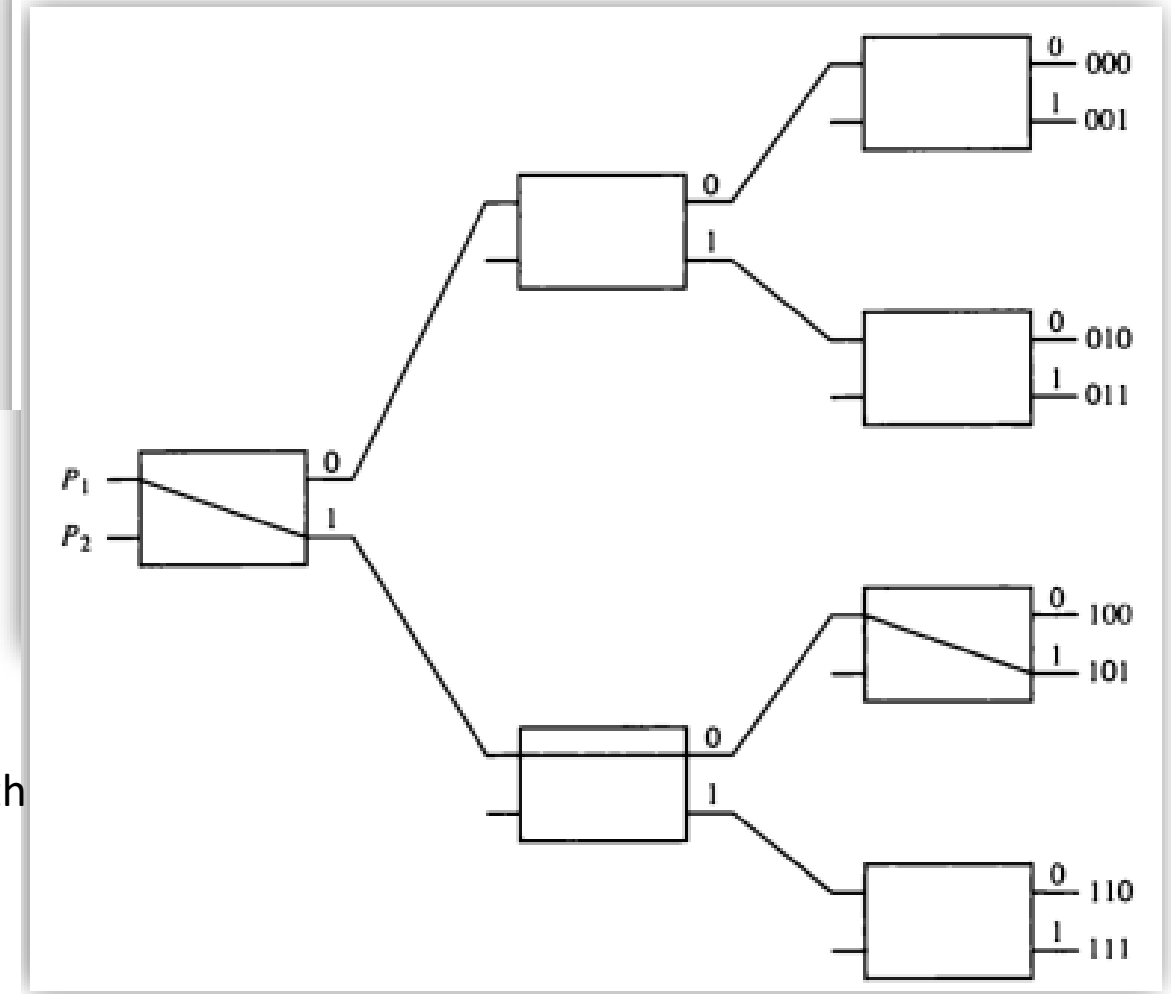
- The basic component of a multistage network is a two-input, two-out interchange switch interchange switch.
- The 2 X 2 switch has two input labeled A and B, and two outputs, labeled 0 and 1.
- The switch has the capability connecting input A to either of the outputs. Terminal B of the switch behaves in a similar fashion.
- If inputs A and B both request the same output terminal only one of them will be connected; the other will be blocked.



Interconnection Structures

4) Multistage Switching Network Interconnection Structures

- To see how this is done, consider the binary tree shown figure
- The two processors P1 and P2 are connected through switches to eight memory modules marked in binary from 000 through 111.
- The path from source to a destination is determined from the binary bits of the destination number.
- Example to connect P1 to memory 101, it is necessary to form a path from P1 to output 1 in first level switch, output 0 in second level switch and output 1 in the third level switch.



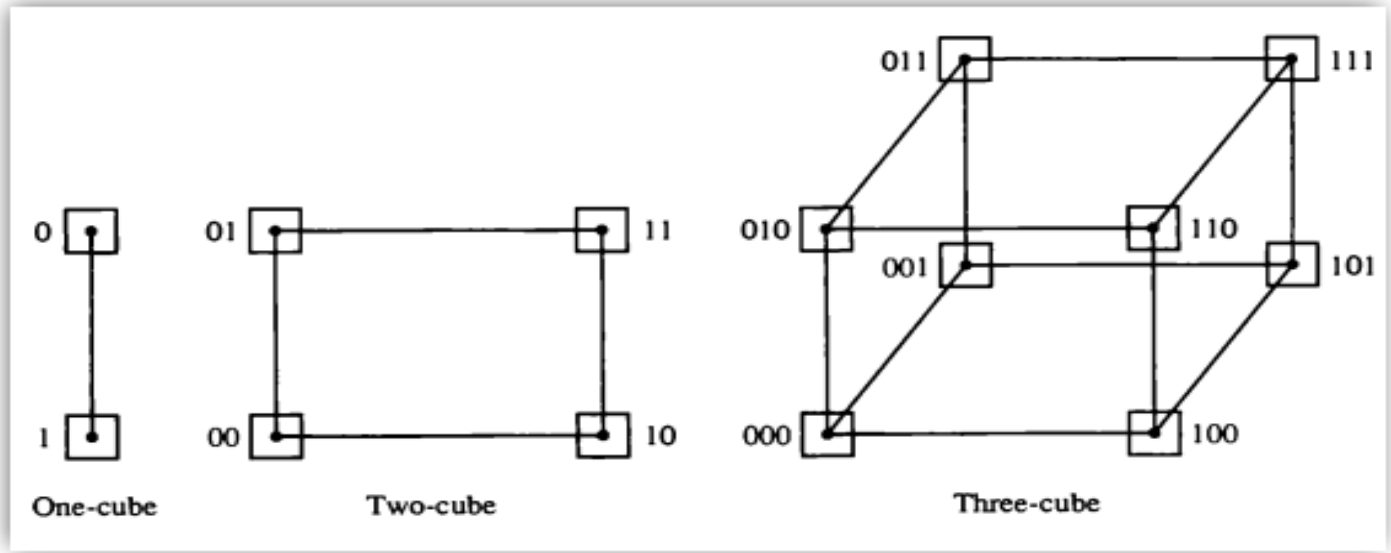
❖ Disadvantages of this network can not connect same output with Both inputs, above example you can not connect P2 with output 1 Because it is reversed for P1

Here P2 is connected with 000,001,010,011

P1 is connected with 100,101,110,111

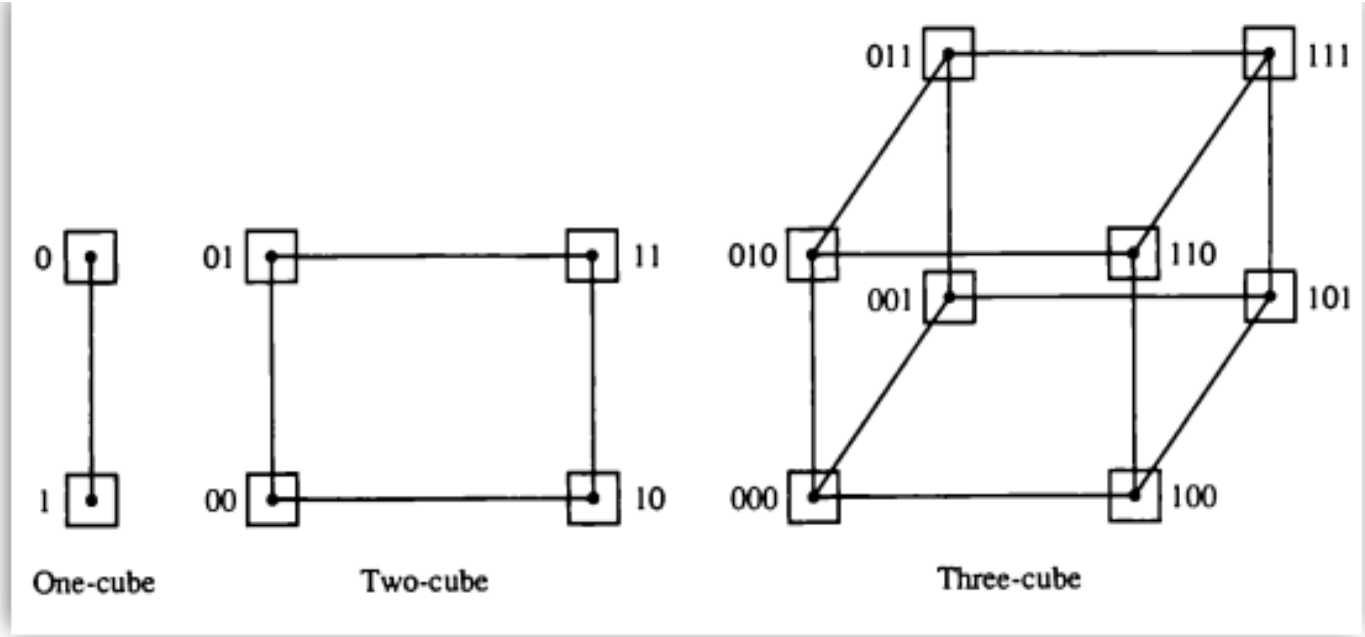
Interconnection Structures

5) Hypercube Interconnection Structures



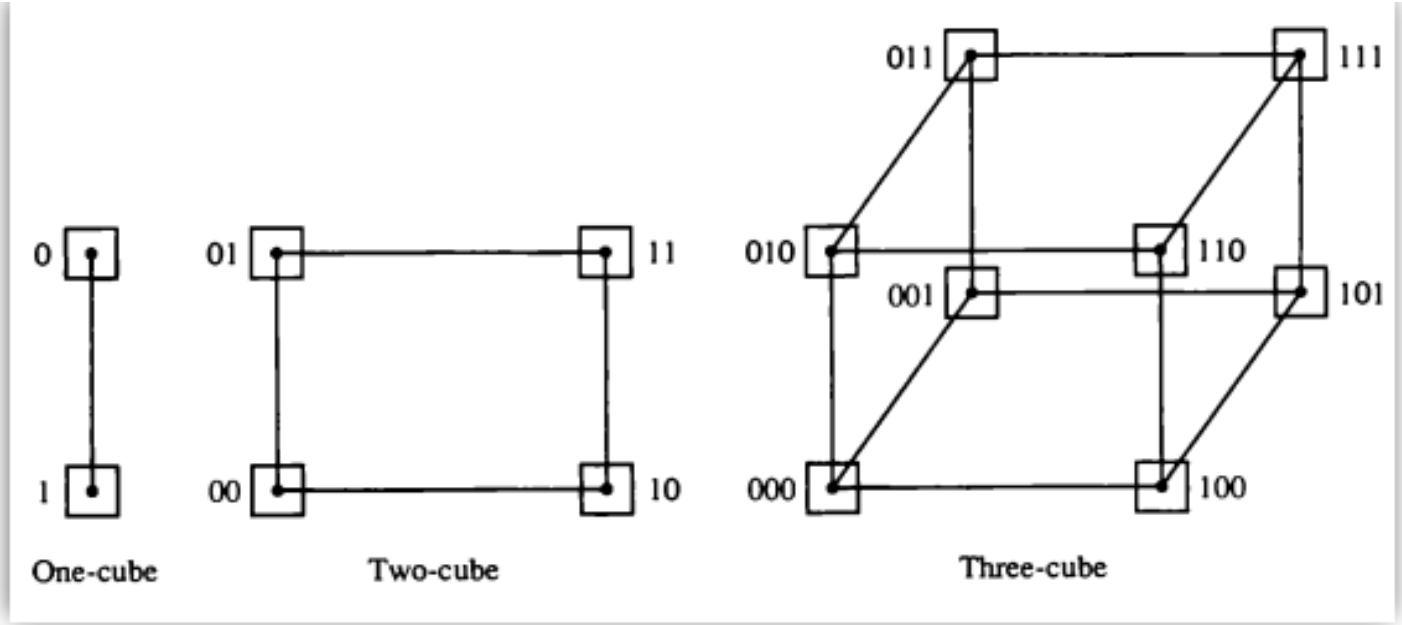
- Each processor forms a node of the cube.
- Each processor has direct communication paths to other neighbor processors. These paths correspond to the edges of the cube.

Interconnection Structures



- A one-cube structure has $n = 1$ and $2^n = 2$. It contains two processors interconnected by a single path.
- A two-cube structure has $2^1 = 2$ and $2^n = 4$. It contains four nodes interconnected as a square.
- A three-cube structure has eight nodes interconnected as a cube.
- An n-cube structure has 2^n nodes with a processor residing in each node.
- Each node is assigned a binary address in such a way that the addresses of two neighbors differ in exactly one bit position.
- For example, the three neighbors of the node with address 100 in a three-cube structure are 000,110, and 101. Each of these binary numbers differs from address 100 by one bit value.

Interconnection Structures



- For example, in a three-cube structure, node 000 can communicate directly with node 001.
- It must cross at least two links to communicate with 011 (from 000 to 001 to 011 or from 000 to 010 to 011).
- It is necessary to go through at least three links to communicate from node 000 to node 111.
- A routing procedure can be developed by computing the exclusive-OR of the source node address with the destination node address.
- For example, in a three-cube structure, a message at 010 going to 001 produces an exclusive- OR of the two addresses equal to 011.
- The message can be sent along the second axis to 000 and then through the third axis to 001.

Inter-processor Arbitration

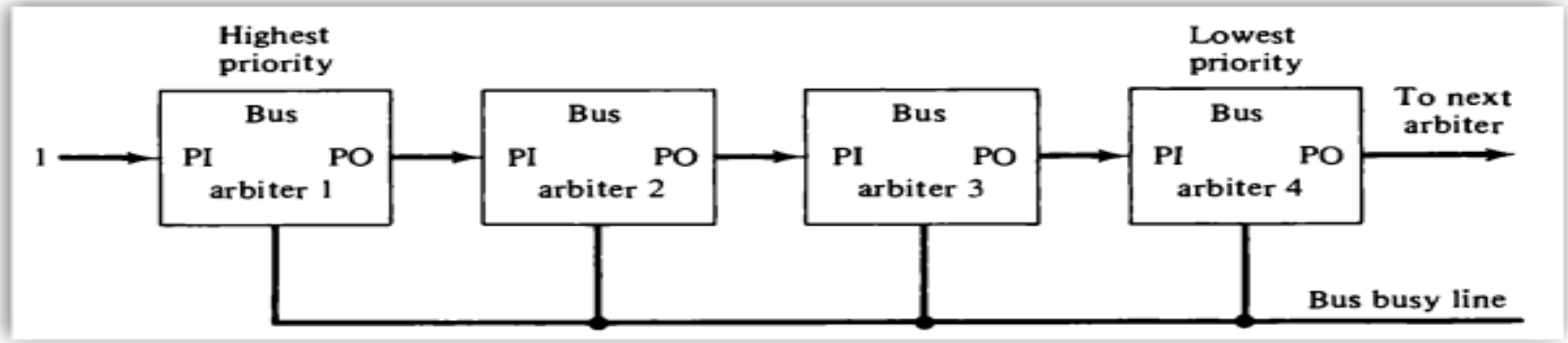
- Arbitration is mainly use **when we have multiprocessor present in the system and when multiple processor puts a request to the bus in order to access the memory** then the **arbiter** is decided **which** Processor to access memory first….

- There are three types of arbitration procedures :

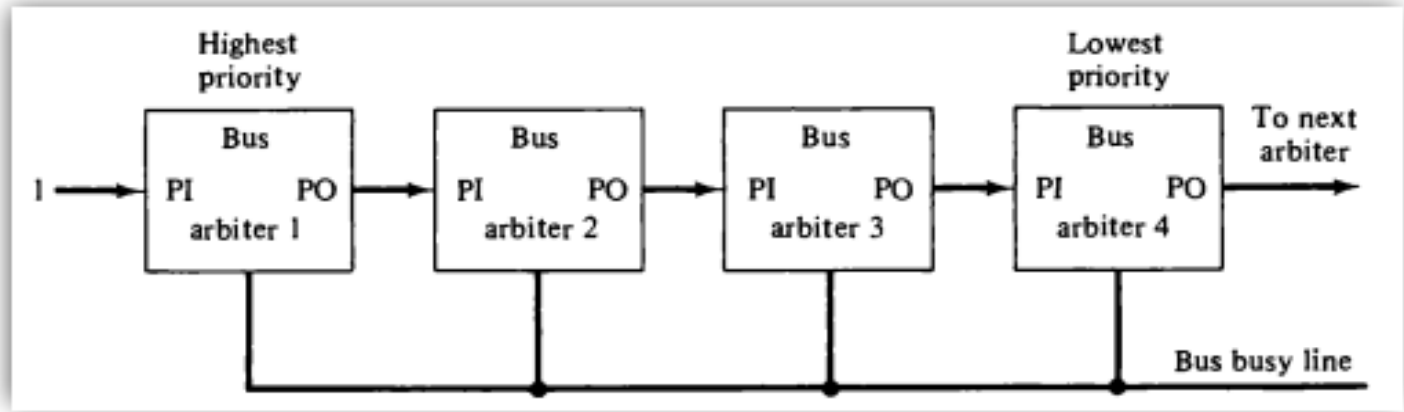
1. Serial (Daisy Chain) Arbitration Procedure
2. Parallel Arbitration Procedure
3. Dynamic Arbitration Algorithms

1. Serial (Daisy Chain) Arbitration Procedure

- Arbitration procedures service all processor requests on the basis of established priorities.
- The processors connected to the system bus are assigned priority according to their position.
- When multiple devices concurrently request the use of the bus, the device with the highest priority is granted access to it.
- It is assumed that each processor has its own bus arbiter logic with priority.
- The priority out (PO) of each arbiter is connected to the priority in (PI) of the next lower priority arbiter.



1. Serial (Daisy Chain) Arbitration Procedure



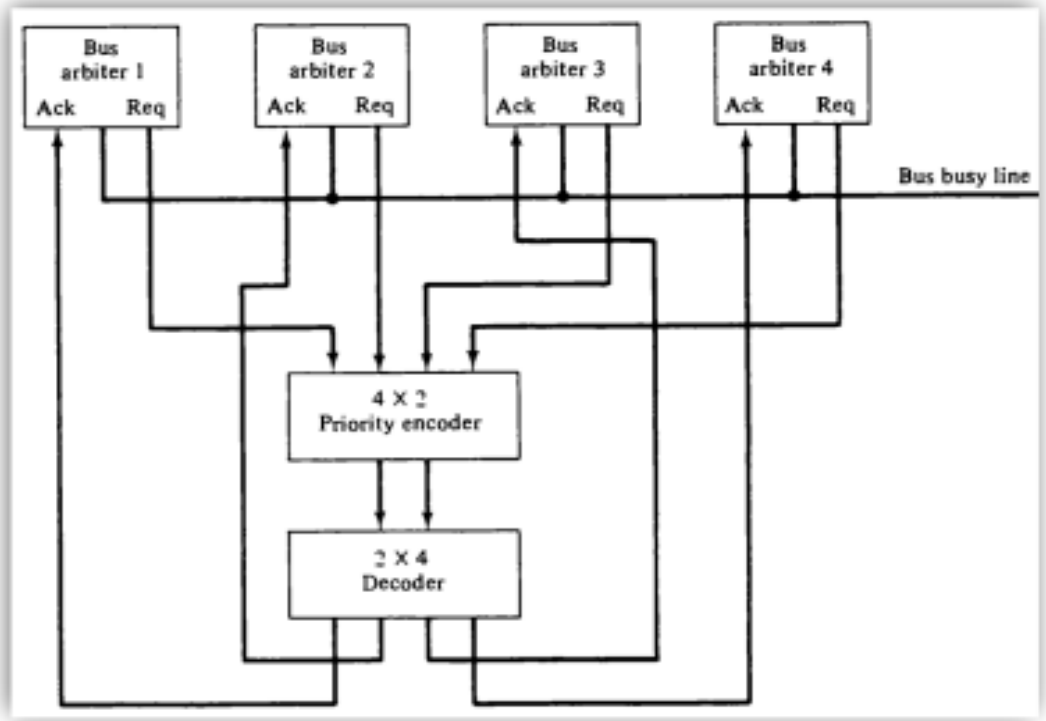
- The PI of the highest-priority unit is maintained at logic 1 value.
- The highest-priority unit in the system will always receive access to the system bus when it requests it.
- The PO output for a particular arbiter is equal to 1 if its PI input is equal to 1 and the processor associated with the arbiter logic is not requesting control of the bus.
- This is the way that priority is passed to the next unit in the chain.
- When processor requests control of the bus and the corresponding arbiter finds its PI input equal to 1, it sets its PO output to 0.
- Thus the processor whose arbiter has a PI = 1 and PO = 0 is the one that is given control of the system bus.
- If the line is inactive, it means that no other processor is using the bus.

2. Parallel Arbitration Procedure

Inputs				Outputs		
I_0	I_1	I_2	I_3	x	y	IST
1	×	×	×	0	0	1
0	1	×	×	0	1	1
0	0	1	×	1	0	1
0	0	0	1	1	1	1
0	0	0	0	×	×	0

Parallel Arbitration Logic

- The parallel bus arbitration technique uses an external priority encoder and a decoder.
- Each bus arbiter in the parallel scheme has a bus request output line and a bus acknowledge input line.
- Each arbiter enables the request line when its processor is requesting access to the system bus.
- The processor takes control of the bus if it acknowledge input line is enabled.
- The bus busy line provides an orderly transfer of control, as in the Daisy chaining case.
- Figure shows the request lines from four arbiters going into a 4 X 2 priority encoder.



- The output of the encoder generates a 2-bit code which represents the highest-priority unit among those requesting the bus.
- The bus priority-in (BPRN) and bus priority-out (BPRO) are used for a daisy-chain connection of bus arbitration circuits.

3. Dynamic Arbitration Algorithms

- **A dynamic priority algorithm gives the system the capability for changing the priority of the devices while the system is in operation.**
- **Time slice** :
- In time-slice there is one time period is given each arbiter in this time period arbiter have to access the system bus if any of the arbiter is fail to access the system bus it means that arbiter have to wait again it time slice.
- **Polling :**
- In Polling all units are connected with Processor
- For example One Processor is connected with 3 memory unit, polling lines are connected with memory unit. then polling to given sequencing address to all memory and access the data for same.
- **Least Recently Used (LRU)**
- The least recently used (LRU) algorithm gives the highest priority to the requesting device that has not used the bus for the longest interval.
- In other word which Processor is Least used is replaced with new one.
- **FIFO**
- In the first-come first-serve scheme, requests are served in the order received.
- To implement this algorithm the bus controller establishes a queue arranged according to the time that the bus requests arrive.
- Each processor must wait for its turn to use the bus on a first-in first-out (FIFO) basis.

Inter-processor Communication and Synchronization

- **INTER-PROCESSOR means** how one Processor will communicate with another Processor.
- Inter Processor communication we used **shared memory (Common memory)** which is used by all the Processor in the system
- Shared memory works like **mail box.**
- **The primary use of the common memory is to act as a message center** similar to **a mailbox,** where each processor can leave messages for other processors and pick up messages intended for it.
- The **sending processor structures a request, a message or a procedure**, and places it in the memory mailbox.
- **Status bits residing in common(Shared) memory** are generally used to indicate the condition of the mailbox, **whether it has meaningful information or not** and for which processor it is intended.
- **Receiving processor can check the mailbox periodically to determine if there are valid messages for it.**
- The response time of this procedure can be **time consuming.**
- A more efficient procedure is for the sending processor to alert the receiving processor directly by means of an **interrupt signal.**

Inter-processor Communication and Synchronization

- **Mailbox is useful to store the messages.**
- **Whenever send Processor wants to send the messages to receiver Processor then sender Processor store the messages into the mailbox.**
- In mailbox **we have one status bit register**, it specifies **the status of mailbox , so that receiver wants to periodically check the mailbox**, if any new message is or not
- Every time receiver Processor to check the status bits to check new message is arrived or not , **it is very time consuming Process and it will handle by interrupts.**
- **Whenever sender Processor send message to the receiver Processor at that time it also send the interrupt to receiver, receiver receives actual message from mail box.**
- **Sometimes sender and receiver access mailbox simultaneously it is major problem then some conflicts occur and this conflicts can be handle by operating system.**

Inter-processor Communication and Synchronization

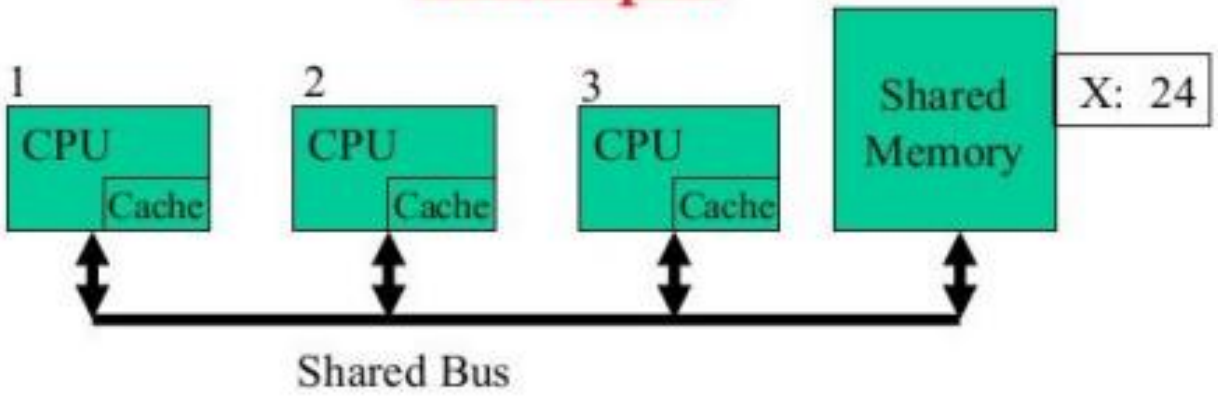
- Best example of synchronization is **Producer consumer Problem.**
- In this Problem we have **2 Processors** 1) Producer 2) Consumer
- Producer Produces a item and places in the Buffer and Consumer collect the item from the **buffer**, but the Producer Produces a item in faster rate and Consumer collect the item from the buffer in slower rate **so number of items store in buffer**.
- **To over come this type of Problem we need the synchronization.**
- Synchronization is handle by two methods, **Critical section and mutual exclusion**
- It is a part of Program where the shared variables are to be access.it is critical session
- While access the shared variable only one Process can enter into critical session it is called mutual execution.

Cache Coherence

Cache Coherence

- In a multiprocessor system, **data inconsistency** may occur among adjacent levels or within the same level of the memory hierarchy. For example, the cache and the main memory may have inconsistent copies of the same data.
- As multiple processors operate in parallel, and independently multiple caches may possess different copies of the same memory block, **this creates cache coherence problem.**
- Cache coherence schemes help to avoid this problem by maintaining a uniform state for each cached block of data.

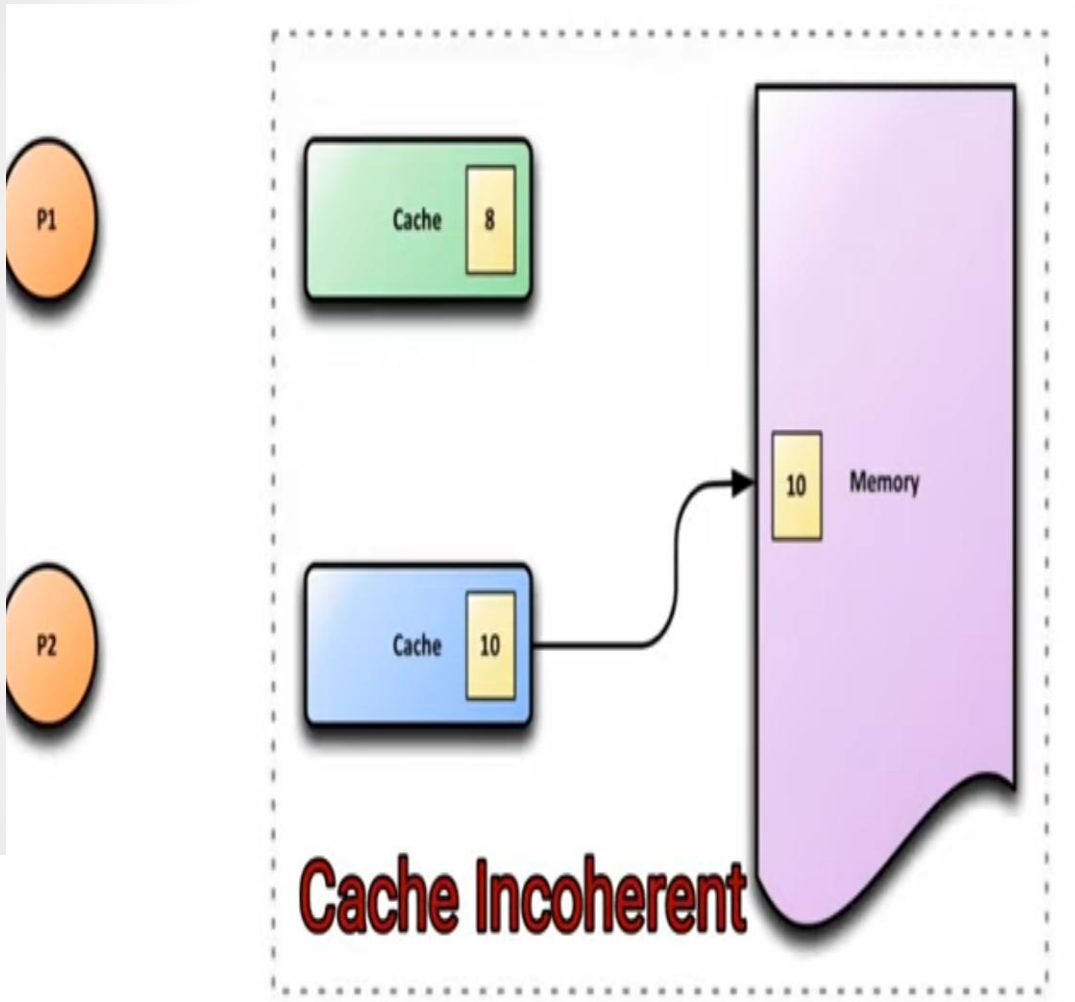
Example

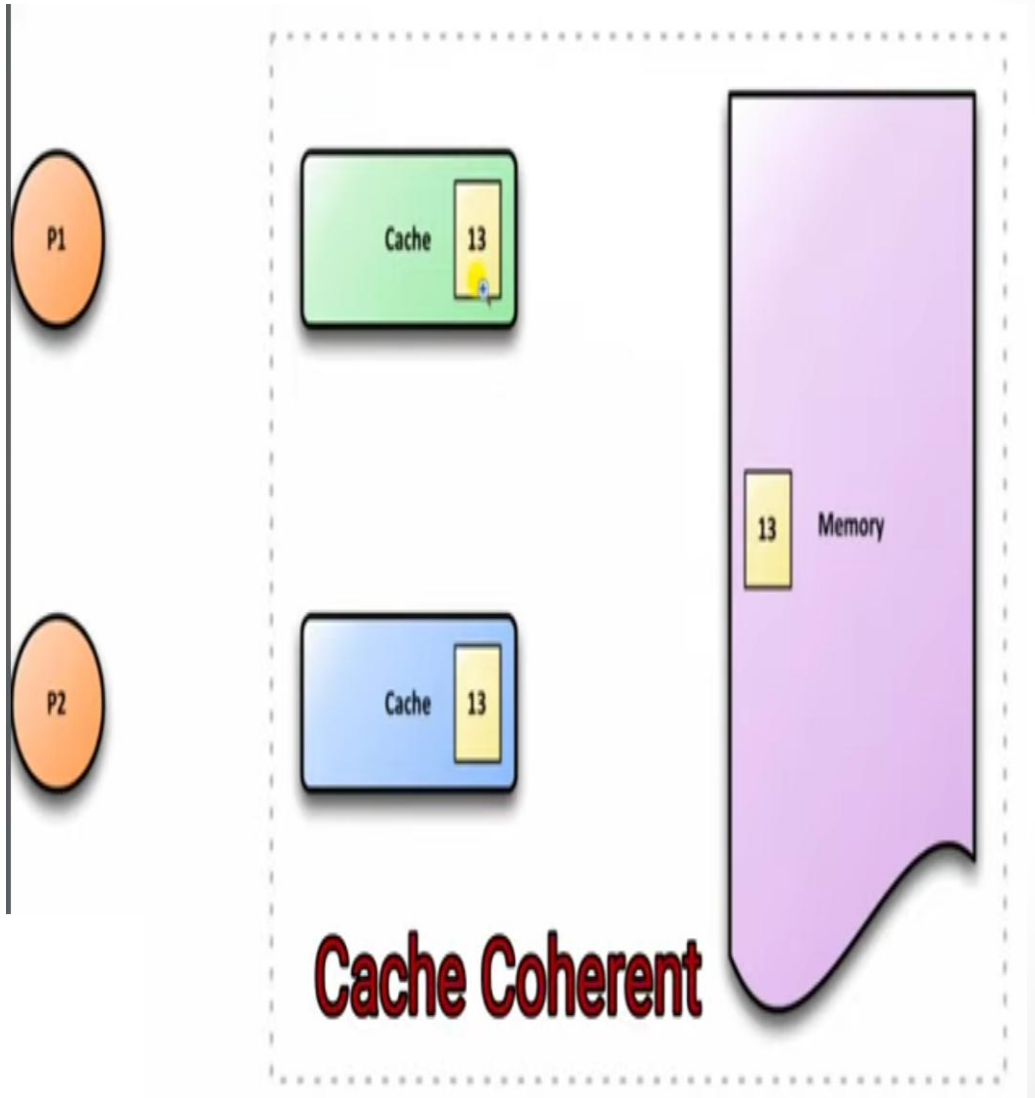


Processor 1 reads X: obtains 24 from memory and caches it
Processor 2 reads X: obtains 24 from memory and caches it
Processor 1 writes 32 to X: its locally cached copy is updated
Processor 3 reads X: what value should it get?
Memory and processor 2 think it is 24
Processor 1 thinks it is 32

Cont...

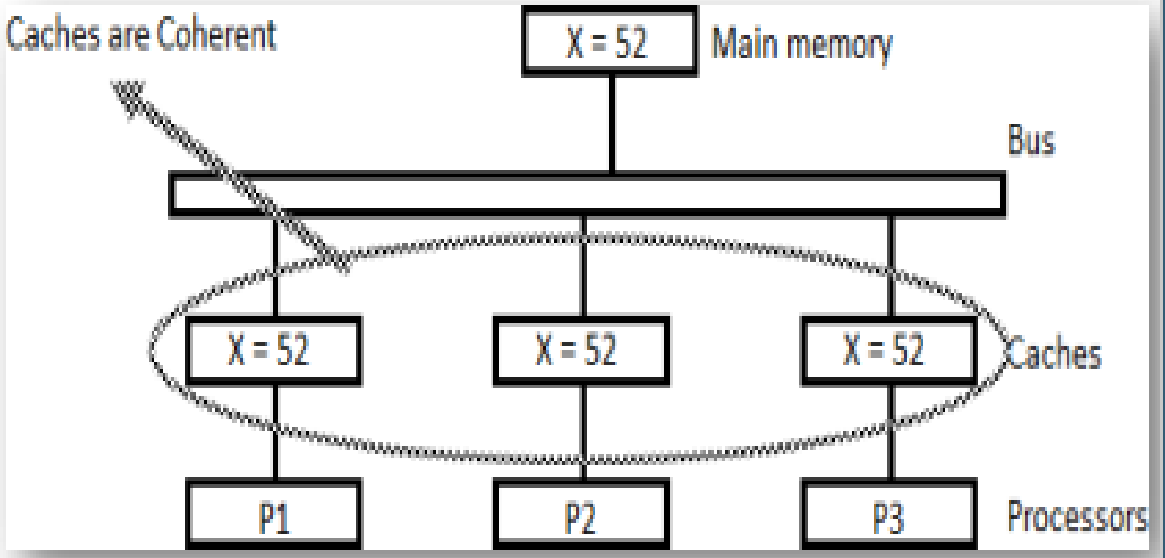
Example Cache coherence and incoherence





Cache write Policy Mechanism

- During the operation an element X from main memory is loaded into the three processors, P1, P2, and P3.
- It is also copied into the private caches of the three processors.
- For simplicity, we assume that X= 52.
- The load on X to the three processors results in consistent copies in the caches and main memory.

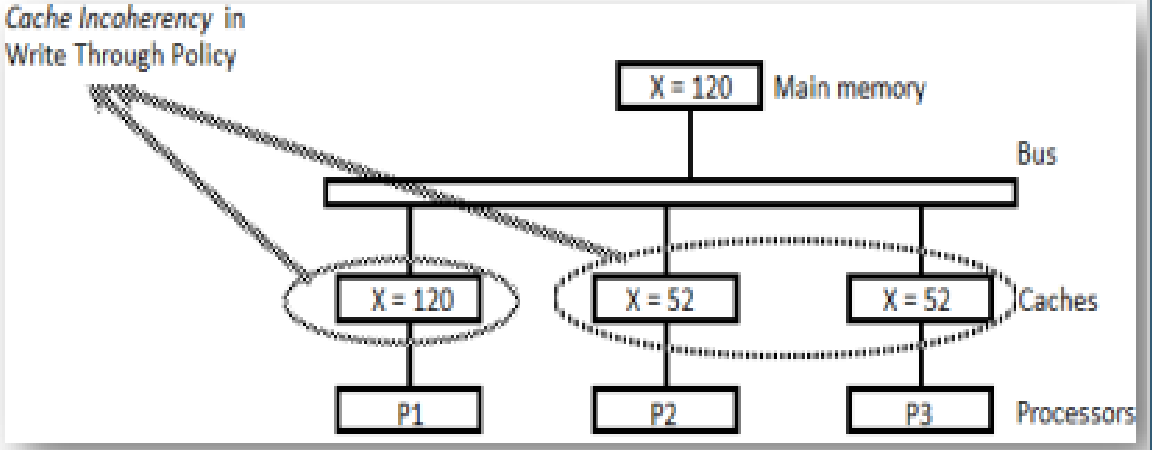


1) Write through policy – all the data written the cache Is also written in memory same time.

2) Write back Policy –when data written the cache, a dirty bit is set for affected block, and it is modified only when block is replace or invalid.

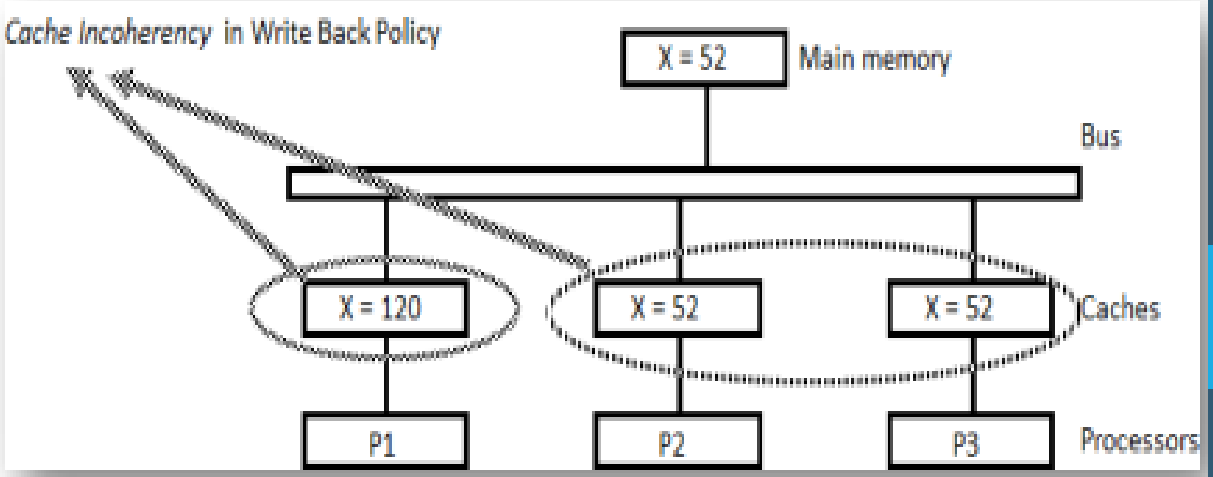
Cache write Policy

- **Write-through policy**
- Here a store to X (of the value of 120) into the cache of processor P1 updates memory to the new value in a write-through policy.
- A write-through policy maintains consistency between memory and the originating cache, but the other two caches are inconsistent since they still hold the old value.



Cont...

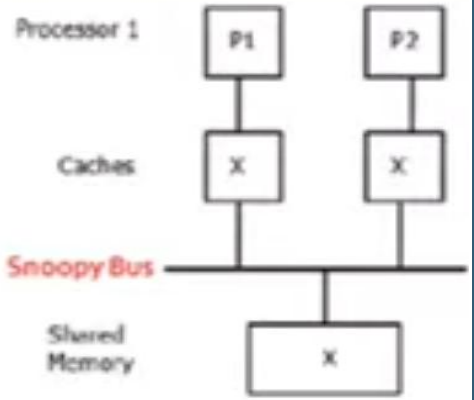
- **Write-back policy**
- In a write-back policy, main memory is not updated at the time of the store.
- The copies in the other two caches and main memory are inconsistent.
- Memory is updated eventually when the modified data in the cache are copied back into memory.



Cache Coherence maintained by Snoopy Cache Controller (Snoopy-Bus Protocol)

Snoopy-Bus Protocol

- Used for bus-based multiprocessor systems
- Transactions on bus are visible to all processors.
- In **Snoopy-Bus protocol**, *each processor's* ***cache controller monitors or snoops on the bus for memory transactions*** *and* ***take*** *proper* ***action to invalidate or update the local cache content*** *if needed*
  - A bus allows all processors in the system to observe ongoing transactions. If a bus transaction threatens the consistent state of a locally cached object, the cache controller can take appropriate actions to invalidate or update the local copy. It is called as snoopy protocol because each cache snoops on the transaction of other caches
- Shortcoming : not scalable
- More scalable solution: 'directory based' coherence schemes



Two Basic Protocols:

Cache Coherence maintained by Snoopy Cache Controller (Snoopy-Bus Protocol)

Using private caches associated with processors tied to a common bus, two approaches are used to maintain cache consistency

1. Write-invalidate:

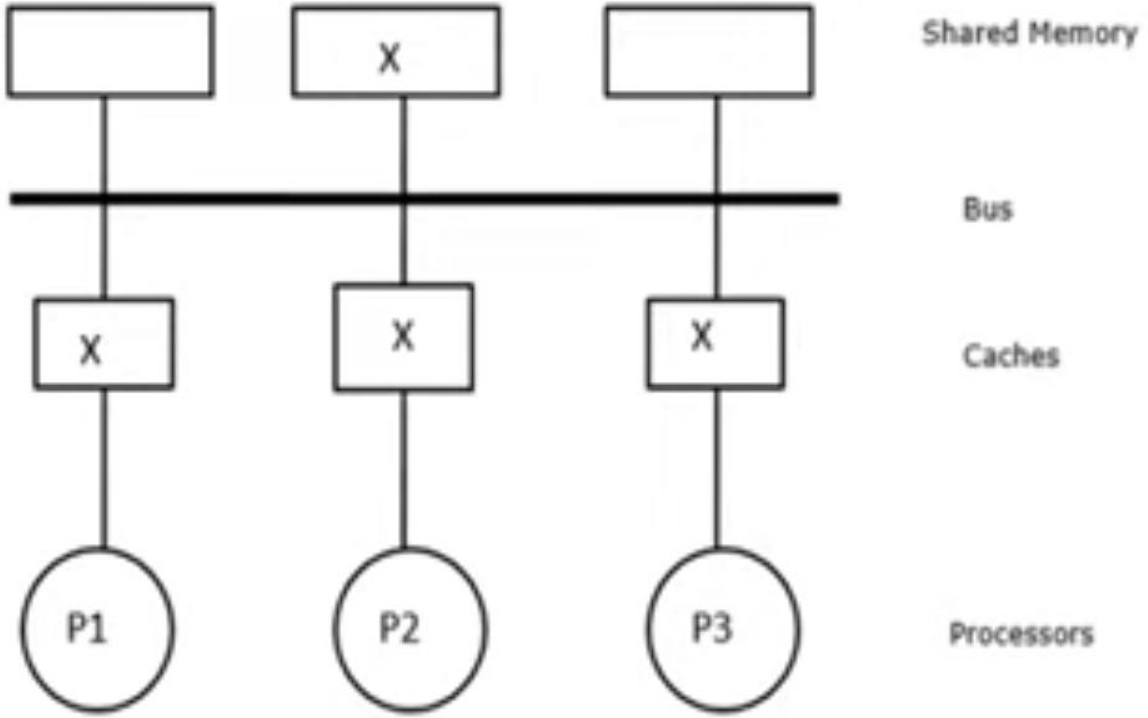
When a local cache copy is modified, **Write-invalidate policy** **it invalidates all remote copies of cache** (invalidated items are sometimes called "dirty")

2. Write-update (Write-broadcast):

When a local cache copy is updated, **Write-update policy** **broadcast a modified value of a data object to all other caches at the time of modification**

Consider Example

- Lets see the write-invalidate and write-update coherence protocols write-through caches
- Consider three processors (Pl, P2, and P3) maintaining consistent copies of block X in their local caches and in the shared-memory module



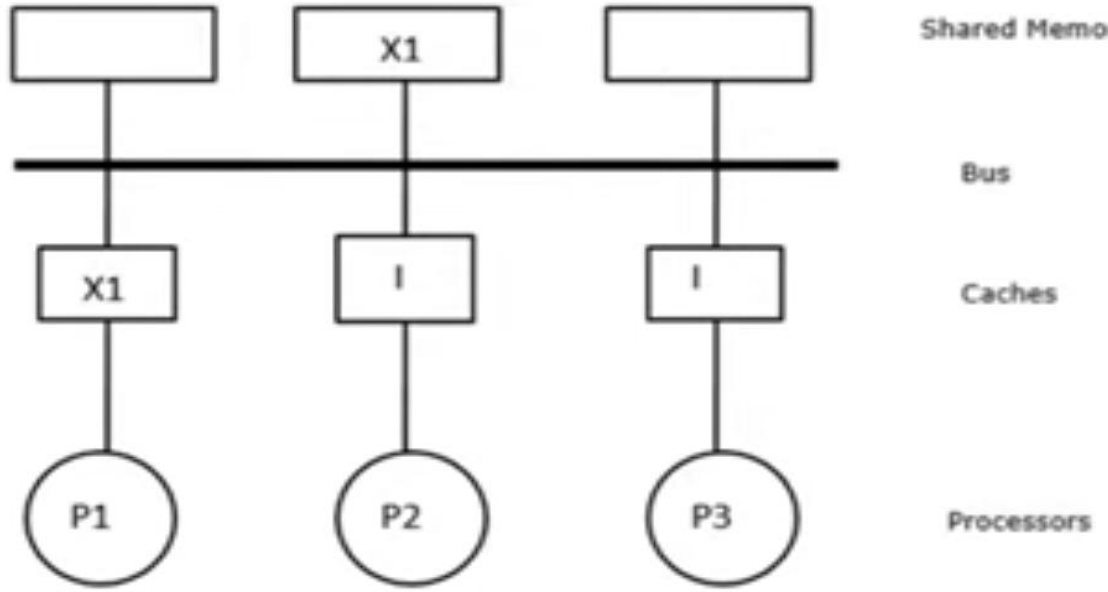
(a) Consistent copies of block X are in shared memory and three processors P1, P2 and P3

Two approaches to maintain cache Consistency

write-invalidate protocol

With Example

- ***Using a write-invalidate protocol,***
  - If processor P1 modifies (writes) its cache from X to X1, then all other copies are **invalidated** via the bus. Invalidated blocks are sometimes called ***dirty***, which means they should not be used.



(b) After write-invalidate operation by P1

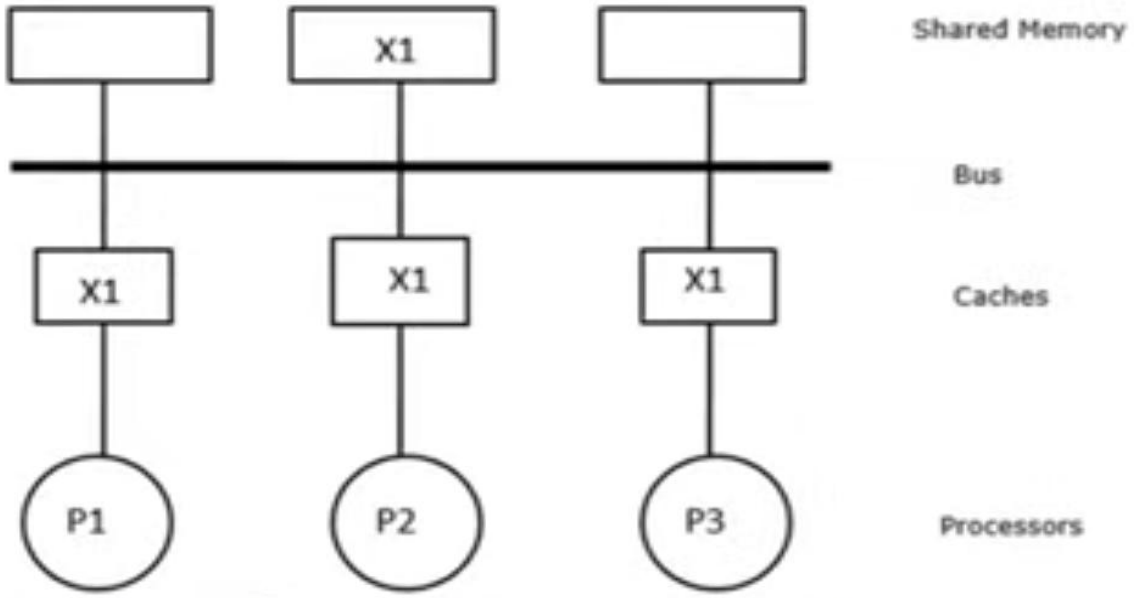
- The memory copy is also updated if **write-through caches** are used.
- In using **write-back caches,** the memory copy is updated later at block replacement time

Two approaches to maintain cache Consistency

write-update protocol

With Example

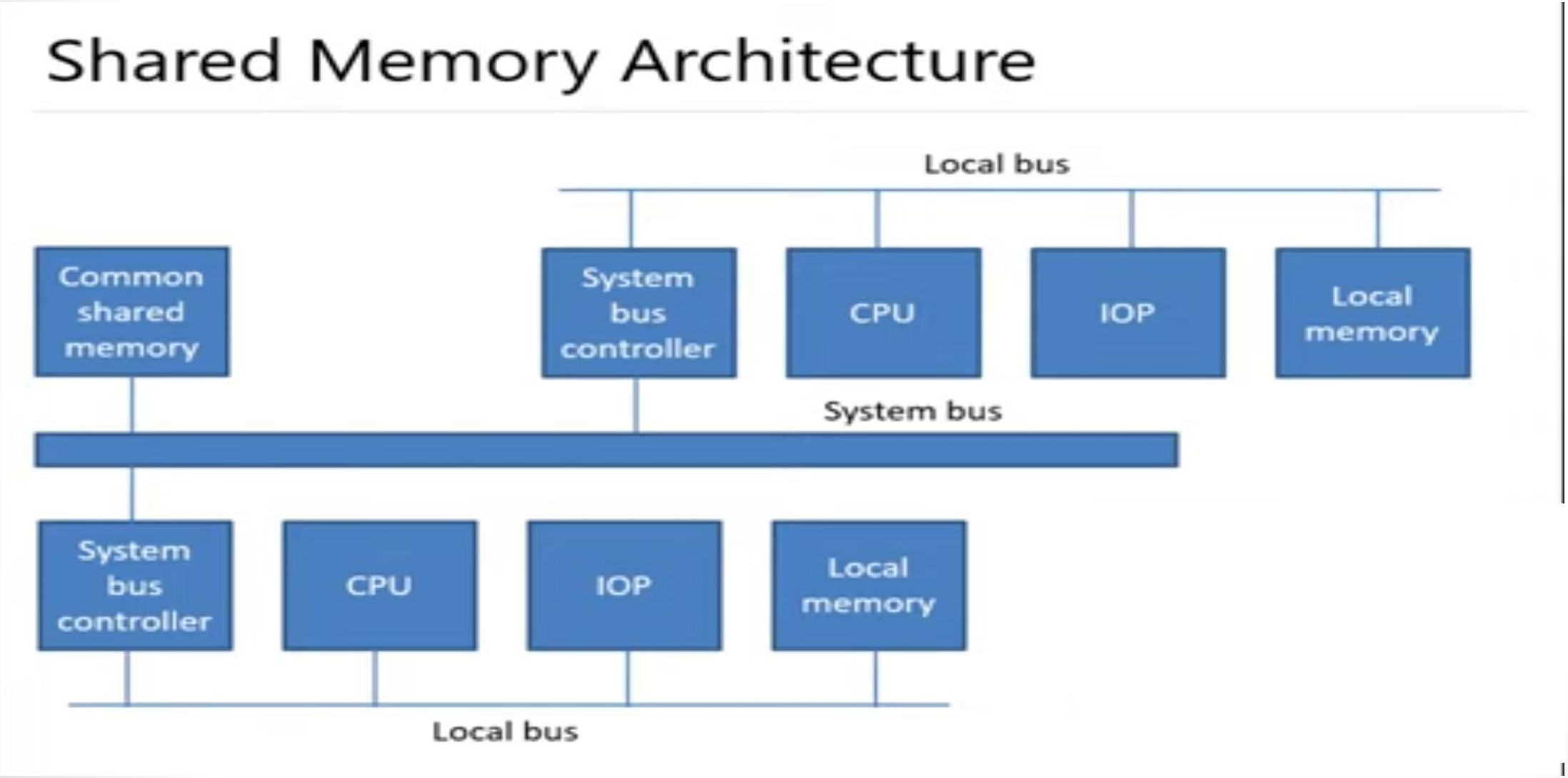
- ***The write-update protocol demands,***
  - The new modified content X1 be **broadcast (update)** to all cache copies via the bus.



(c) After write-update operation by P1

- The memory copy is also updated if **write-through caches** are used.
- In using **write-back caches,** the memory copy is updated later at block replacement time

Shared Memory Multiprocessors



QUESTIONS ASKED IN GTU EXAM

Q.1 Discuss cache coherence problem in detail.(3 times)
Q.2 Explain multiport memory and crossbar switch with reference to interconnection structures in multiprocessors.
Q.3 Discuss multistage switching network with neat diagrams.
Q.4 Write about Time-shared common bus interconnection structure.
Q.5 Write a note on inter process communication and synchronization.